

Students: Burair Haidar, Syed Sualeh

Hardware Wiring Guide

Use jumper wires to connect the following pins.

1. Motor Direction Pins (From code comments to shield)

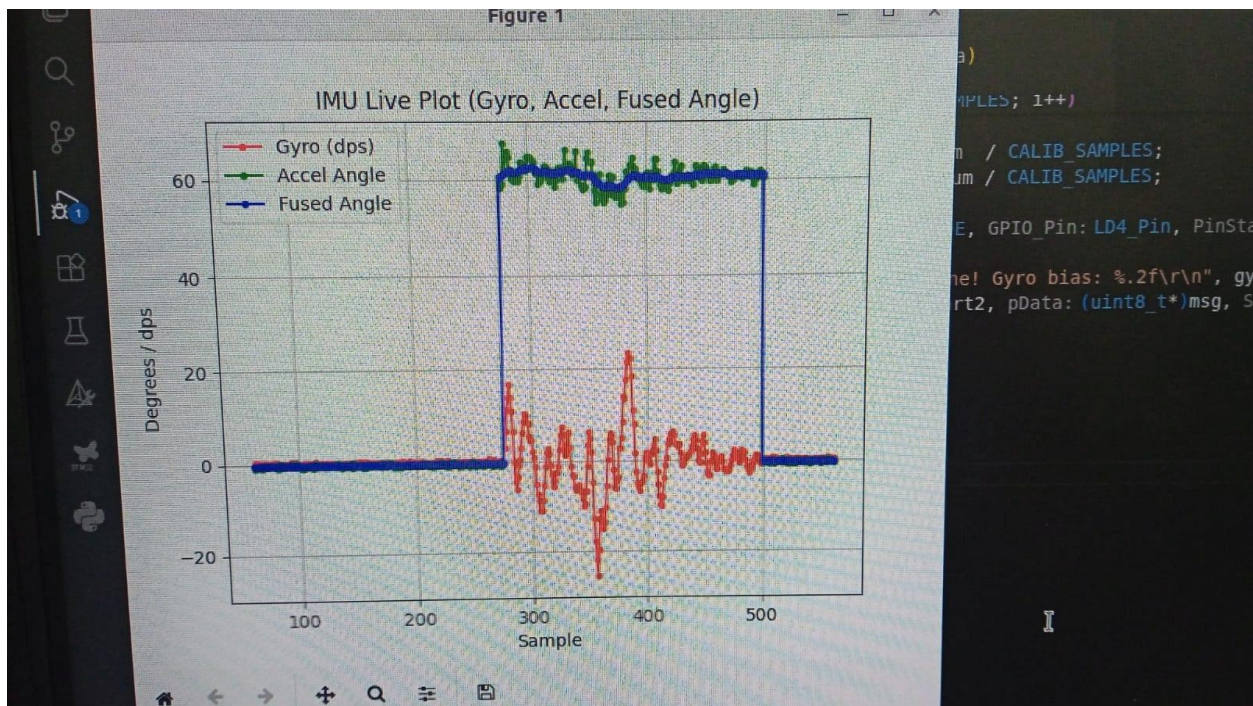
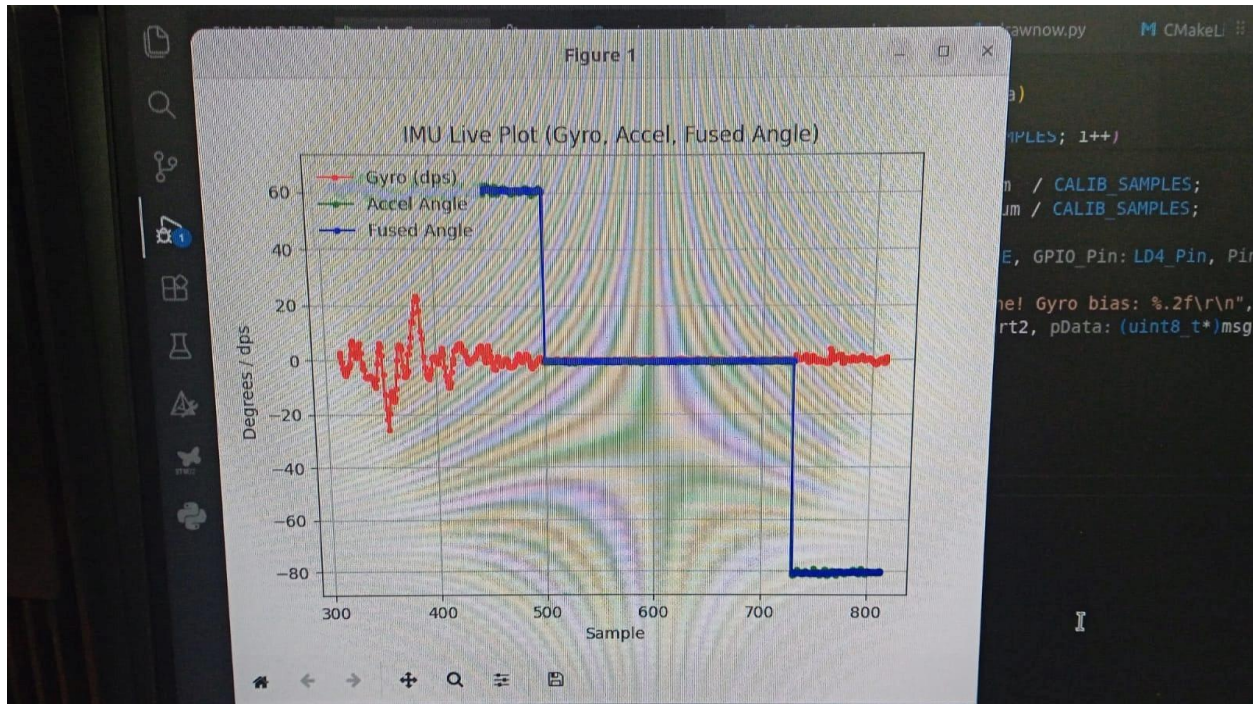
STM32F3 Pin	Keyestudio Shield Pin	Function
PE8	D8	Right Motor Direction 1
PE12	D12	Right Motor Direction 2
PE7	D7	Left Motor Direction 1
PE6	D6	Left Motor Direction 2

2. Motor Speed (PWM) Pins *Note: The test code uses TIM3_CHANNEL_1 for the right motor and TIM3_CHANNEL_2 for the left motor. Depending on how you generated the code in STM32CubeIDE, these timer channels could be mapped to different physical pins. You must check your .ioc file (Device Configuration Tool) to see which physical pins are assigned to TIM3 CH1 and CH2.*

STM32F3 Pin	Keyestudio Shield Pin	Function
[Your TIM3_CH1 Pin] (Often PA6 or PC6)	D10	Right Motor PWM (Speed)
[Your TIM3_CH2 Pin] (Often PA7 or PA4)	D9	Left Motor PWM (Speed)

RX—PA2

TASK 1 RESULT:



TASK 2 RESULTS WILL SHOWN TO THE INSTRUCTOR IN LAB

```

/* USER CODE BEGIN Header */
/**
 * ****
 * ****
 *
 * @file      : main.c
 * @brief     : Self-Balancing Robot -
Main Integration
 * ****
 * ****
 *
 */
/* USER CODE END Header */
/* Includes -----
-----*/
#include "main.h"

/* Private includes -----
-----*/
/* USER CODE BEGIN Includes */
#include <math.h>
#include <stdio.h>
#include <string.h>
#include <stdarg.h>
/* USER CODE END Includes */

/* Private typedef -----
-----*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define -----
-----*/
/* USER CODE BEGIN PD */

/* ---- Sampling Time ---- */
#define DT 0.005f // 200Hz -> 1/200
= 0.005s

/* ---- PID Gains (SET TO ZERO FOR NOW
TO FIND SETPOINT) ---- */

```

```
#define KP 40.0f
#define KI 0.5f
#define KD 2.0f

/* ---- Setpoint ---- */
// Target angle. You will change this
// number after reading the serial monitor!
#define SETPOINT 0.0f

/* ---- Output Limits ---- */
#define PID_OUT_MAX 999.0f // TIM3
// period = 999, max PWM
#define PID_OUT_MIN -999.0f

/* ---- Integral Anti-Windup Limit ---- */
#define INTEGRAL_MAX 200.0f
#define INTEGRAL_MIN -200.0f

/* ---- Complementary Filter Weights ----
*/
#define COMP_GYRO_WEIGHT 0.98f
#define COMP_ACC_WEIGHT 0.02f

/* USER CODE END PD */

/* Private macro -----
-----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----
-----*/
I2C_HandleTypeDef hi2c1;
I2C_HandleTypeDef hi2c2;

SPI_HandleTypeDef hspi1;

TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim3;

UART_HandleTypeDef huart2;
```

```
PCD_HandleTypeDef hpcd_USB_FS;
```

```
/* USER CODE BEGIN PV */
/* ---- Angle Estimation Variables ---- */
volatile float shared_angle = 0.0f; //
Bridge between ISR and main
volatile float shared_pid_out = 0.0f; //
PID output for display
volatile uint8_t display_flag = 0;
```

```
// For display/debug in main loop
float gyro_rate = 0.0f;
float acc_angle = 0.0f;
/* USER CODE END PV */
```

```
/* Private function prototypes -----
-----*/
```

```
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_SPI1_Init(void);
static void MX_TIM3_Init(void);
static void MX_USB_PCD_Init(void);
static void MX_I2C2_Init(void);
static void MX_TIM2_Init(void);
static void MX_USART2_UART_Init(void);
/* USER CODE BEGIN PFP */
```

```
/* USER CODE END PFP */
```

```
/* Private user code -----
-----*/
```

```
/* USER CODE BEGIN 0 */
```

```
/*
=====
=====
* I3G4250D (Gyroscope) SPI Read/Write
*
=====
===== */
void I3G_WriteReg(uint8_t reg_addr,
uint8_t data) {
```

```
uint8_t tx_buffer[2];
tx_buffer[0] = reg_addr & 0x7F; // Bit 7 =
0 for WRITE
tx_buffer[1] = data;
```

```
// Use CS pin (PE3)
HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_3, GPIO_PIN_RESET); // CS
LOW
HAL_SPI_Transmit(&hspi1, tx_buffer, 2,
HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_3, GPIO_PIN_SET); // CS
HIGH
}
```

```
uint8_t I3G_ReadReg(uint8_t reg_addr) {
uint8_t tx_data = reg_addr | 0x80; // Bit 7
= 1 for READ
uint8_t rx_data;
```

```
HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_3, GPIO_PIN_RESET);
HAL_SPI_Transmit(&hspi1, &tx_data, 1,
HAL_MAX_DELAY);
HAL_SPI_Receive(&hspi1, &rx_data, 1,
HAL_MAX_DELAY);
HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_3, GPIO_PIN_SET);
```

```
return rx_data;
}
```

```
/*
=====
=====
* Motor Control Functions (Using Verified
GPIOE Pins)
*
=====
===== */
```

```
void Motor_Left_Forward(uint16_t speed)
{
```

```

    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_7, GPIO_PIN_SET); // PE7 =
HIGH
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_6, GPIO_PIN_RESET); // PE6 =
LOW
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_2, speed);
}

```

```

void Motor_Left_Backward(uint16_t
speed){
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_7, GPIO_PIN_RESET); // PE7 =
LOW
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_6, GPIO_PIN_SET); // PE6 =
HIGH
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_2, speed);
}

```

```

void Motor_Left_Stop(void) {
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_7, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_6, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_2, 0);
}

```

```

void Motor_Right_Forward(uint16_t
speed){
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_8, GPIO_PIN_SET); // PE8 =
HIGH
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_12, GPIO_PIN_RESET); // PE12
= LOW
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_1, speed);
}

```

```

void Motor_Right_Backward(uint16_t
speed) {
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_8, GPIO_PIN_RESET); // PE8 =
LOW
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_12, GPIO_PIN_SET); // PE12 =
HIGH
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_1, speed);
}

```

```

void Motor_Right_Stop(void) {
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_8, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE,
GPIO_PIN_12, GPIO_PIN_SET);
    __HAL_TIM_SET_COMPARE(&htim3,
TIM_CHANNEL_1, 0);
}

```

```

/*
=====
=====
* Drive logic based on PID
*
=====
===== */

```

```

void Motors_Drive(float pid_output) {
    uint16_t pwm_val;

    if (pid_output > 0) {
        if (pid_output > PID_OUT_MAX)
pid_output = PID_OUT_MAX;
        pwm_val = (uint16_t)pid_output;
        Motor_Left_Forward(pwm_val);
        Motor_Right_Forward(pwm_val);
    }
    else if (pid_output < 0) {
        float abs_out = -pid_output;
        if (abs_out > PID_OUT_MAX) abs_out =
PID_OUT_MAX;
        pwm_val = (uint16_t)abs_out;
        Motor_Left_Backward(pwm_val);
    }
}

```

```

        Motor_Right_Backward(pwm_val);
    }
    else {
        Motor_Left_Stop();
        Motor_Right_Stop();
    }
}

/* UART Print Helper */
void cout(const char *fmt, ...) {
    char buffer[128];
    va_list args;
    va_start(args, fmt);
    int l = vsnprintf(buffer, sizeof(buffer),
fmt, args);
    va_end(args);
    if (l > 0) {
        HAL_UART_Transmit(&huart2,
(uint8_t*)buffer, l, HAL_MAX_DELAY);
    }
}

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{

/* USER CODE BEGIN 1 */

/* USER CODE END 1 */

/* MCU Configuration-----
-----*/

/* Reset of all peripherals, Initializes the
Flash interface and the Systick. */
HAL_Init();

/* USER CODE BEGIN Init */

```

```

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_I2C1_Init();
MX_SPI1_Init();
MX_TIM3_Init();
MX_USB_PCD_Init();
MX_I2C2_Init();
MX_TIM2_Init();
MX_USART2_UART_Init();
/* USER CODE BEGIN 2 */
// ---- Initialize Gyroscope (I3G4250D via
SPI) ----
// CTRL_REG1 = 0x20: 200Hz ODR,
normal mode, XYZ enable
I3G_WriteReg(0x20, 0x4F);

// ---- Initialize Accelerometer
(LSM303DLHC via I2C) ----
// CTRL_REG1_A = 0x20, value 0x57 =
100Hz, normal mode, XYZ enabled
uint8_t acc_init[2] = {0x20, 0x57};
HAL_I2C_Master_Transmit(&hi2c1, 0x32,
acc_init, 2, 50);

// ---- Start PWM on both channels for
motor speed control ----
HAL_TIM_PWM_Start(&htim3,
TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim3,
TIM_CHANNEL_2);

// Initialize motors stopped
Motor_Left_Stop();
Motor_Right_Stop();

```

```

// ---- Start TIM2 interrupt for 200Hz
control loop ----
HAL_TIM_Base_Start_IT(&htim2);

// Small delay to let sensors settle
HAL_Delay(100);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    if (display_flag == 1) {
        display_flag = 0;

        // Float printing bypass for embedded
C
        int shared_angle_i =
(int)shared_angle;
        int shared_angle_f =
(int)(fabsf(shared_angle - shared_angle_i)
* 100);

        int acc_angle_i = (int)acc_angle;
        int acc_angle_f =
(int)(fabsf(acc_angle - acc_angle_i) *
100);

        int gyro_rate_i = (int)gyro_rate;
        int gyro_rate_f = (int)(fabsf(gyro_rate -
gyro_rate_i) * 100);

        int shared_pid_out_i =
(int)shared_pid_out;
        int shared_pid_out_f =
(int)(fabsf(shared_pid_out -
shared_pid_out_i) * 100);

        // Print to Serial Monitor
        cout("Angle: %d.%02d | Acc:
%d.%02d | Gyro: %d.%02d | PID:
%d.%02d\r\n",
            shared_angle_i, shared_angle_f,
            acc_angle_i, acc_angle_f,

```

```

        gyro_rate_i, gyro_rate_f,
        shared_pid_out_i,
        shared_pid_out_f);
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**
 * @brief GPIO Initialization Function
 * @param None
 * @retval None
 */
static void MX_GPIO_Init(void)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    /* GPIO Ports Clock Enable */
    __HAL_RCC_GPIOE_CLK_ENABLE();
    __HAL_RCC_GPIOC_CLK_ENABLE();
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    __HAL_RCC_GPIOB_CLK_ENABLE();

    GPIO_InitStruct.Mode =
GPIO_MODE_EVT_RISING;

    /*
=====
=====
* TIM2 ISR Callback — runs at 200 Hz
(every 5ms)
*
=====
===== */
void
HAL_TIM_PeriodElapsedCallback(TIM_Ha
ndleTypeDef *htim) {

```

```

if (htim->Instance == TIM2) {

    static float tilt_angle = 0.0f;
    static float integral = 0.0f;
    static float prev_error = 0.0f;
    static int uart_counter = 0;

    /* --- STEP 1: GYRO --- */
    uint8_t y_low =
I3G_ReadReg(0x2A); // OUT_Y_L
    uint8_t y_high =
I3G_ReadReg(0x2B); // OUT_Y_H
    int16_t raw_gyro_y = (int16_t)((y_high
<< 8) | y_low);
    float gyro_y_dps = (float)raw_gyro_y *
0.00875f;

    /* --- STEP 2: ACCEL --- */
    uint8_t acc_reg = 0x28 | 0x80;
    uint8_t acc_buf[6] = {0};
    HAL_I2C_Master_Transmit(&hi2c1,
0x32, &acc_reg, 1, 10);
    HAL_I2C_Master_Receive(&hi2c1,
0x33, acc_buf, 6, 10);
    int16_t acc_raw_x =
(int16_t)((acc_buf[1] << 8) | acc_buf[0]);
    int16_t acc_raw_z =
(int16_t)((acc_buf[5] << 8) | acc_buf[4]);
    float acc_angle_deg =
atan2f((float)acc_raw_x, (float)acc_raw_z)
* (180.0f / 3.14159f);

    /* --- STEP 3: COMP FILTER --- */
    tilt_angle = COMP_GYRO_WEIGHT *
(tilt_angle + gyro_y_dps * DT)
    + COMP_ACC_WEIGHT *
acc_angle_deg;

    /* --- STEP 4: PID --- */

```

```

float error = SETPOINT - tilt_angle;

float p_term = KP * error;

integral += KI * error * DT;
if (integral > INTEGRAL_MAX) integral =
INTEGRAL_MAX;
if (integral < INTEGRAL_MIN) integral =
INTEGRAL_MIN;

float derivative = (error - prev_error) /
DT;
float d_term = KD * derivative;
prev_error = error;

float pid_output = p_term + integral +
d_term;

if (pid_output > PID_OUT_MAX)
pid_output = PID_OUT_MAX;
if (pid_output < PID_OUT_MIN)
pid_output = PID_OUT_MIN;

/* --- STEP 5: DRIVE --- */
Motors_Drive(pid_output);

/* --- STEP 6: SHARED VARS --- */
shared_angle = tilt_angle;
shared_pid_out = pid_output;
gyro_rate = gyro_y_dps;
acc_angle = acc_angle_deg;

/* --- STEP 7: UART THROTTLE (10Hz) --
- */
if (++uart_counter >= 20) {
    display_flag = 1;
    uart_counter = 0;
}

```

EVALUATION QUESTIONS:

a) Role of each PID term

A PID controller works by continuously correcting the error between what you want and what the system is actually doing, but each term approaches that correction differently. The proportional term reacts immediately to the current error. If the system is far from the desired value, it responds strongly; if it is close, the response becomes gentle. This makes it responsible for the main “push” toward the target.

The integral term looks at how the error has behaved over time. Even if the system is only slightly off but stays that way for a long period, this term builds up and forces the system to correct that persistent offset. Without it, the system might settle close to the desired value but never exactly reach it.

The derivative term focuses on how the error is changing. It essentially predicts where the system is heading and applies a braking effect if things are moving too fast. This helps reduce overshooting and makes the system more stable and smoother in its response.

(b) Effect of changing K_p , K_i , and K_d

When you increase the proportional gain, the system reacts more aggressively to errors, which makes it respond faster. However, if it is too high, the system can start oscillating or even become unstable because it keeps over-correcting.

Increasing the integral gain helps eliminate any steady-state error, ensuring the system eventually reaches the exact desired value. The downside is that too much integral action can make the system sluggish and may introduce oscillations because it keeps accumulating past errors.

The derivative gain improves stability by damping the system’s response. It reduces overshoot and smooths out oscillations. However, if it is set too high, it can make the system overly sensitive to noise, especially in real-world signals, leading to erratic behavior.

(c) Challenges in initial tuning and strategy for fine tuning

In the initial tuning phase, one of the main challenges is that the system often behaves unpredictably. For example, a small change in gains can suddenly cause large oscillations, especially in our case, the system response was too aggressive even on slightest angles. Another issue is that all three parameters are interconnected, so adjusting one affects the others, making it difficult to isolate their individual effects, so motor rotation was also a problem, no initial response from shield was observed.

For final tuning, a more structured approach will be better. A small amount of derivative gain will be added to stabilize and reduce oscillations. Finally, the integral term can be introduced carefully to remove any remaining steady state error. Final Filtering will involve making very small adjustments, testing repeatedly, and observing key behaviors like overshoot, settling time, and stability rather than just reacting to one issue at a time.

(d) Limitations of a simple PID in real-world systems

A simple PID controller works well for many systems, but it has several limitations in real world applications. One major issue is that it assumes the system behaves in a relatively predictable and linear way, which is often not true. Real systems can be non-linear, have delays, and changing dynamics that a basic PID cannot handle.

Another limitation is sensitivity to noise. Also, PID controllers do not adapt on their own; once tuned, they may perform poorly if system conditions change.

Finally, for complex systems with multiple interacting variables, a single PID controller is often not enough. More advanced control strategies, such as adaptive control or state-space methods, are usually needed to achieve better performance.